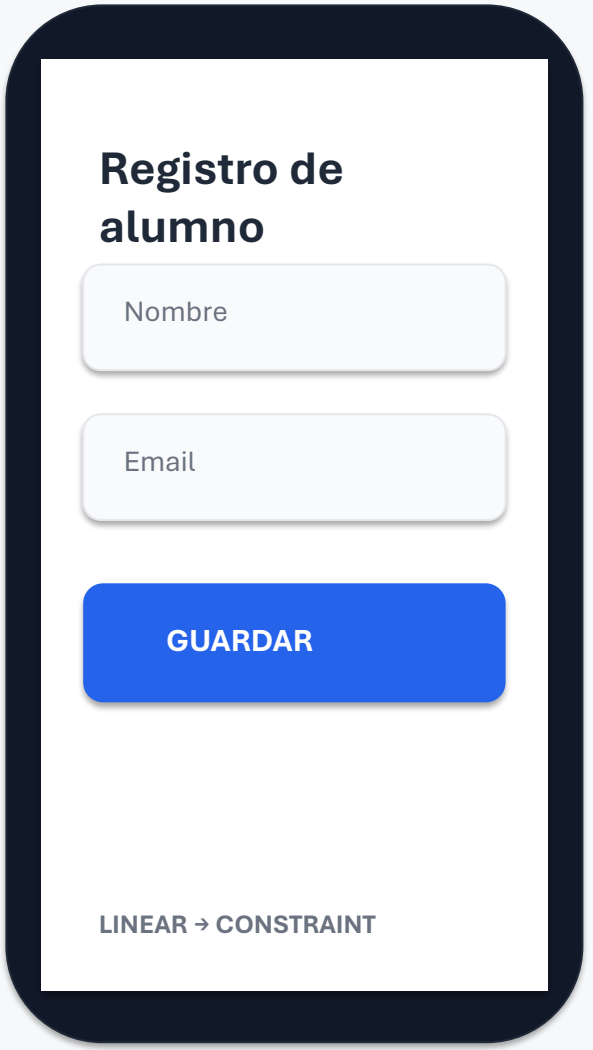
Práctica guiada: LinearLayout y ConstraintLayout

Android Studio · XML · Nivel inicial

Construiremos la misma pantalla de registro de dos formas.

Objetivo: comprender cómo se colocan las vistas en un LinearLayout y cómo se relacionan mediante constraints en un ConstraintLayout.



Docente: José Manuel Villar Ferradal

1. Crear el proyecto

Partimos de cero para que todos sigan los mismos pasos.

- Abrir Android Studio → New Project.
- Elegir Empty Views Activity (no Jetpack Compose).
- Nombre: LayoutsPractica.
- Lenguaje: Kotlin.
- Finalizar y esperar a que Gradle termine.

¿Dónde trabajaremos?

app → res → layout → activity_main.xml

Consejo docente
Usar la vista Code o Split para relacionar XML y resultado visual.

2. Preparar la primera pantalla: LinearLayout

Un LinearLayout coloca sus hijos uno detrás de otro.

- Sustituye el contenido de activity_main.xml.
- orientation="vertical" apila las vistas de arriba abajo.
- padding="24dp" separa el contenido de los bordes.
- match_parent hace que el layout ocupe todo el espacio disponible.

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="24dp">

<!-- Aquí añadiremos las vistas -->

</LinearLayout>
```

3. Añadir los controles al LinearLayout

Título + dos campos + botón.

- Añade estas vistas dentro del LinearLayout.
- El orden del XML determina el orden vertical.
- wrap_content: la vista mide lo que necesita.
- match_parent: ocupa el ancho disponible.

```
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Registro de alumno"
android:textSize="24sp"
android:textStyle="bold" />

<EditText
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Nombre" />

<EditText
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Email"
android:inputType="textEmailAddress" />

<Button
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Guardar" />
```

4. Mejorar el espaciado

Los márgenes pertenecen a cada vista; el padding pertenece al contenedor.

- Añade layout_marginTop="16dp" al primer EditText.
- Repite el margen en el segundo EditText y en el Button.
- Ejecuta la app y observa la secuencia vertical.
- Prueba a cambiar orientation a horizontal y explica qué ocurre.

Fragmento a añadir

```
android:layout_marginTop="16dp"
```

¿Podemos colocar fácilmente el botón pegado al fondo manteniendo los campos arriba?

5. Crear la versión con ConstraintLayout

Ahora las vistas se posicionan mediante relaciones entre ellas.

- Crea un nuevo archivo: res/layout/activity_constraint.xml.
- El contenedor ya no tiene orientation.
- Cada vista debe quedar suficientemente restringida en horizontal y vertical.
- Usaremos IDs para relacionar unas vistas con otras.

```
<androidx.constraintlayout.widget.ConstraintLayout

xmlns:android="http://schemas.android.com/apk/res/android"
     xmlns:app="http://schemas.android.com/apk/res-auto"
             android:layout_width="match_parent"
            android:layout_height="match_parent"
                  android:padding="24dp">

         <!-- Las vistas necesitarán constraints -->

   </androidx.constraintlayout.widget.ConstraintLayout>
```

6. Título y primer campo con constraints

Primero anclamos elementos al padre y después entre sí.

- Start/End fijan los lados horizontales.
- Top del título se une al Top del padre.
- El campo Nombre se coloca debajo del título.
- 0dp + constraints laterales significa “match constraints”: ocupar el espacio entre ambos límites.

```
<TextView
    android:id="@+id/tvTitulo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Registro de alumno"
    android:textSize="24sp"
    android:textStyle="bold"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent" />

<EditText
    android:id="@+id/etNombre"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:hint="Nombre"
    android:layout_marginTop="16dp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toBottomOf="@id/tvTitulo" />
```

7. Completar ConstraintLayout

Relacionamos cada nueva vista con la anterior.

- Email depende verticalmente de Nombre.
- Guardar depende verticalmente de Email.
- Todos quedan alineados con Start/End del padre.
- Ejecuta Preview y comprueba que no aparezcan avisos de constraints ausentes.

```
<EditText
    android:id="@+id/etEmail"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:hint="Email"
    android:inputType="textEmailAddress"
    android:layout_marginTop="16dp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toBottomOf="@id/etNombre" />

<Button
    android:id="@+id/btnGuardar"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="Guardar"
    android:layout_marginTop="16dp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toBottomOf="@id/etEmail" />
```

8. ¿Cómo mostramos la segunda pantalla?

Para una práctica sencilla, podemos reutilizar MainActivity.

- Abre MainActivity.kt.
- Cambia temporalmente activity_main por activity_constraint.
- Ejecuta la aplicación.
- Después vuelve a activity_main para comparar ambos resultados.

```
// MainActivity.kt
override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

    setContentView(R.layout.activity_constraint)
                }
```

Importante: aquí solo cambiamos el layout que infla la Activity; todavía no estamos creando navegación entre pantallas.

9. LinearLayout vs ConstraintLayout

Misma interfaz, distinta estrategia de colocación.

LinearLayout

- Orden secuencial.
- Muy sencillo para filas o columnas.
- orientation controla la dirección.
- Puede requerir layouts anidados para diseños complejos.

ConstraintLayout

- Relaciones entre vistas.
- Más flexible para interfaces complejas.
- Usa constraints horizontales y verticales.
- 0dp suele significar match constraints.

Idea clave: no hay un “ganador” universal; elegimos el contenedor según la estructura de la interfaz.

10. Reto individual

Modifica la versión ConstraintLayout sin copiar una solución.

- Añade un TextView debajo del botón con el texto “Datos pendientes de guardar”.
- Haz que quede centrado horizontalmente.
- Deja 24dp de separación respecto al botón.
- Añade un segundo botón “Limpiar” a la derecha de “Guardar”.
- Comprueba que todos los elementos tengan constraints suficientes.

Pistas

- Usa un id para cada botón.
- Relaciona Start/End entre botones y padre.
- Para centrar un elemento, puedes restringir Start y End al padre.